

Transport Interface Reference

Wire protocol and message schemas for Cross Guard message transports

Overview

Cross Guard supports four transport providers for relaying messages between Mattermost instances: [NATS](#), Azure Queue Storage, Azure Blob Storage (batched WAL), and Azure Service Bus. Each provider has different delivery guarantees, latency characteristics, and infrastructure requirements. The plugin integrates with Mattermost's shared-channels framework, so outbound messages enter the plugin through the framework's sync hooks (`OnSharedChannelsSyncMsg` , `OnSharedChannelsAttachmentSyncMsg` , `OnSharedChannelsProfileImageSyncMsg` , `OnSharedChannelsPing`), and inbound messages exit back into the framework via `ReceiveSharedChannelSyncMsg` . Sync content (posts, users, reactions, statuses, membership changes, acknowledgements, mention transforms) is wrapped in an XML envelope (or JSON, per the connection's `message_format` setting) and delivered through the transport. File attachments and profile images do not ride the XML envelope; they flow through a separate file-upload channel (`UploadFile` / `WatchFiles` on `QueueProvider`) with metadata carried as `X-Crossguard-*` headers on the blob.

Transport Providers

Aspect	NATS	Azure Queue Storage	Azure Blob Storage	Azure Service Bus
Delivery Model	Pub/sub (push)	Queue polling (pull, default 5s interval)	Batched WAL blob (60s flush, lease-based read)	PeekLock pull (32-msg batch, 5s max wait)
Delivery Guarantee	At-most-once on the transport; shared-channels framework	At-least-once	At-least-once	At-least-once (PeekLock with server-side DeliveryCount)

Aspect	NATS	Azure Queue Storage	Azure Blob Storage	Azure Service Bus
	retries on hook failure			
Message Size Limit	No plugin-enforced limit (NATS server default 1 MB applies)	48,000 bytes (plugin-enforced; before Base64 to the 64 KB Azure Queue wire limit)	Bounded by batch blob size (many messages per blob)	192,000 bytes default; 256 KiB Standard tier ceiling, 100 MiB on Premium
File Transfer	JetStream Object Store	Azure Blob Storage (paired container, default 15s poll interval)	Same container as WAL (deferred file blobs)	Azure Blob Storage sidecar (default 15s poll, independent credentials)
Encryption	TLS (optional)	TLS always (Azure endpoint)	TLS always (Azure endpoint)	TLS always (AMQP over TLS)
Authentication	Token, credentials file, or mTLS	Shared key, or Service Principal (Azure AD client secret)	Shared key, or Service Principal (Azure AD client secret)	SAS connection string, or Service Principal (Azure AD client secret)
Message Encoding	Raw bytes	Base64 over wire	Length-prefixed batch within the blob	AMQP message body (raw bytes)
Auto-Provisioning	No (server must exist)	Yes in shared-key mode (queue and container auto-created); no in Service Principal mode (pre-provision required)	Yes in shared-key mode (container auto-created); no in Service Principal mode (pre-provision required)	No (queue must be pre-created with chosen LockDuration and MaxDeliveryCount)

NATS Transport Summary

Sync Transport	NATS core publish/subscribe
File Transport	NATS JetStream Object Store (bucket: <code>crossguard-files</code>)
Wire Format	XML or JSON (UTF-8) for sync envelopes; binary for files
Subject Prefix	<code>crossguard.</code> (required for all subjects)
Subject Pattern	<code>crossguard.{domain}</code> , e.g. <code>crossguard.high</code> , <code>crossguard.low</code>
Retry Policy	The shared-channels framework controls retries via hook-return semantics. A failed <code>OnSharedChannelsSyncMsg</code> call leaves the framework cursor unadvanced, so the framework retries on its own schedule.
Inbound Concurrency	Single-active receiver per inbound connection across the cluster (KV TTL lease). The handler is deliberately serial so the inbound sequencer can dedup and reorder envelopes correctly.

Data Flow

The wire-level data flow is the same regardless of transport provider; only the publish and subscribe primitives change. The diagrams below describe the framework-to-framework path for a single sync envelope.

Outbound (Local to Remote)

```

User action in Mattermost
  |
  v
Shared-channels framework fires OnSharedChannelsSyncMsg
  |
  v
Resolve outbound connection via connNameForRemote(rc.RemoteId)
  |
  v
buildOutboundEnvelopes: split SyncMsg into one envelope per post
(plus an optional metadata envelope for non-post content)
  |
  v
publishToOutboundConn stamps Epoch and Sequence on every envelope
  |

```

```

    v
MarshalEnvelope serializes (XML by default; JSON per message_format)
    |
    v
QueueProvider.Publish (NATS subject, Azure queue, Service Bus queue,
or Azure Blob WAL append)
```

Attachments and profile images are emitted from `OnSharedChannelsAttachmentSyncMsg` and `OnSharedChannelsProfileImageSyncMsg` via `QueueProvider.UploadFile` rather than through the XML envelope path. They carry their metadata as `x-Crossguard-*` headers on the file blob.

Inbound (Remote to Local)

```

Active receiver subscribes (single-active election via KV TTL lease,
45s TTL with 15s renewal)
    |
    v
QueueProvider.Subscribe delivers raw bytes to processInboundMessage
    |
    v
UnmarshalEnvelope auto-detects format (first non-whitespace byte:
< means XML, { means JSON; leading UTF-8 BOM on XML is stripped)
    |
    v
inboundSequencer.Admit:
    * drop duplicates by (Epoch, ConnName, ChannelId, Sequence)
    * buffer short gaps for bounded reorder window
    * audit lost ranges when gap-deadline ticker fires
    |
    v
rewriteChannelIDs maps source-side IDs to local channel IDs
    |
    v
ReceiveSharedChannelSyncMsg hands the SyncMsg back to the framework,
which performs the actual post create/update/delete, user upsert,
reaction apply, etc.
    |
    v
Cursor checkpoint ticker persists per-channel sequence cursors to KV
every 5 seconds so leadership handoff resumes cleanly
```

File-channel events (`watchFiles`) follow a parallel inbound path: the handler reads the `x-Crossguard-Kind` header to discriminate attachments from profile images, resolves the team and channel via the same rewrite logic, and calls into the framework's attachment / profile-image upsert APIs.

QueueProvider Interface

Every transport implements the `QueueProvider` interface defined in `server/provider.go` . The interface is intentionally small: a publish / subscribe pair for sync envelopes, a parallel pair for file uploads, a `MaxMessageSize` hint that the splitter respects, an `IsConnected` health probe for the ping hook, and a `close` . Anything provider-specific (queue creation, lease management, batch flushing, dead-letter handling) lives behind these methods.

```
type QueueProvider interface {
    Publish(ctx context.Context, data []byte) error
    Subscribe(ctx context.Context, handler func(data []byte) error) error
    UploadFile(ctx context.Context, key string, data []byte, headers map[string]string) error
    WatchFiles(ctx context.Context, handler func(key string, data []byte, headers map[string]string) error) error
    MaxMessageSize() int
    IsConnected() bool
    Close() error
}
```

Publish

Sends a fully marshalled `TransportEnvelope` (XML or JSON bytes) over the transport. The caller has already applied `Epoch` and `Sequence` .

Subscribe

Delivers incoming envelope bytes to the inbound pipeline. The handler may return an error to defer acknowledgement so the transport retries (Azure Queue dequeue lease, Service Bus PeekLock abandon, etc.).

UploadFile

Carries raw file bytes (`attachment` or `profile_image`) with metadata headers. The `key` follows a stable pattern: `attachment/{postID}/{fileID}` or `profile_image/{userID}` .

WatchFiles

Delivers each new file blob to a handler with the same key and headers the sender used. The handler discriminates the file kind from the `x-Crossguard-Kind` header.

MaxMessageSize	Hard upper bound on the byte length of a single <code>Publish</code> argument. <code>0</code> means "no practical limit"; the plugin treats anything <code>> 0</code> as a budget the envelope splitter must respect.
IsConnected	Cheap health probe used by <code>OnSharedChannelsPing</code> to surface outbound-unhealthy / inbound-unhealthy audit events without forcing a publish.
Close	Releases any provider-side resources (NATS subscriptions, Azure SDK clients, background poll goroutines). Idempotent.
NATS	<code>MaxMessageSize() == 0</code> (no plugin-enforced cap; NATS server default applies)
Azure Queue	<code>MaxMessageSize() == 48000</code> bytes (the plugin caps below 64 KB so Base64 expansion fits the 64 KB Azure Queue wire limit)
Azure Blob	<code>MaxMessageSize() == 0</code> (the WAL buffer multiplexes many envelopes per batch blob; the cap is the blob size, not the envelope size)
Azure Service Bus	<code>MaxMessageSize() == max_message_size_bytes</code> (default 192000; configurable up to 100 MiB on Premium tier)

Message Envelope

`sync_msg` and `test` envelopes ride this schema on the wire. `attachment` and `profile_image` uploads do not use this envelope at all (see [File-Upload Headers](#) below). The outbound connection's `message_format` setting selects the serialization (XML or JSON); inbound is auto-detected by the first non-whitespace byte (`<` for XML, `{` for JSON; a leading UTF-8 BOM on XML is stripped before detection).

JSON field names exactly mirror the XML element and attribute names defined in `schema/crossguard.xsd`, so the two serializations are field-for-field equivalent. The reference Go type is `TransportEnvelope` in `server/transport.go`.

ENVELOPE	CrossGuardEnvelope
Schema	

version

int, required, attribute. Wire-format version, always `1` today. Carried as an XML attribute on the root `<CrossGuardEnvelope>` element; in JSON, a top-level field.

type

string, required, attribute. One of the four [envelope types](#). Determines which payload field is present (or, for `attachment` / `profile_image`, that this envelope is not used at all).

ConnName

string, required. Outbound connection name on the sender (URL-safe slug, max 96 chars). The receiver uses this to scope per-channel sequence numbers and to dispatch audit events.

Timestamp

string, required. UTC timestamp in RFC 3339 format (e.g. `2026-03-27T14:30:00Z`). Defaults to `time.Now().UTC()` at marshal time when the caller leaves it empty.

Epoch

string, required on sender-originated envelopes. 26-character Mattermost ID (`[a-z0-9]{26}`) identifying the sender process generation, so the receiver can correlate sequence numbers across restarts. Stamped on both `sync_msg` and `test` envelopes.

Sequence

unsigned 64-bit int, required on `sync_msg` only. Per- (`ConnName`, `ChannelId`) monotonic counter within an epoch. Lets the receiver detect duplicates and reorder short gaps. Absent on `test` envelopes (no channel scope).

TeamName

string, required. Sender's team name (URL-safe slug, max 96 chars). On `test` envelopes this carries the connection name as a placeholder so the field passes XSD validation; the receiver ignores team and channel scope for `type=test`.

ChannelName

string, required. Sender's channel name (URL-safe slug, max 96 chars). On `test` envelopes this carries the connection name as a placeholder.

SyncMsg

object, optional. Present when `type` is `sync_msg`. Carries the shared-channels sync payload. See [SyncMsg Payload](#).

TestID

string, optional. Present when `type` is `test`. A 26-character Mattermost ID generated by the test-connection handler; the receiver echoes it into its audit log.

XML Example (`sync_msg` with one post)

Wire output is a single line (no indentation between elements); indented here for readability.

```
<?xml version="1.0" encoding="UTF-8"?>
<CrossGuardEnvelope version="1" type="sync_msg">
  <ConnName>high-side</ConnName>
  <Timestamp>2026-04-12T09:00:00Z</Timestamp>
  <Epoch>epoch01aaaaaaaaaaaaaaaaaaaa</Epoch>
  <Sequence>1</Sequence>
  <TeamName>team-a</TeamName>
  <ChannelName>general</ChannelName>
  <SyncMsg>...</SyncMsg>
</CrossGuardEnvelope>
```

JSON Example (same envelope)

```
{
  "version": 1,
  "type": "sync_msg",
  "ConnName": "high-side",
  "Timestamp": "2026-04-12T09:00:00Z",
  "Epoch": "epoch01aaaaaaaaaaaaaaaaaaaa",
  "Sequence": 1,
  "TeamName": "team-a",
  "ChannelName": "general",
  "SyncMsg": { ... }
}
```

XML Example (test)

```
<?xml version="1.0" encoding="UTF-8"?>
<CrossGuardEnvelope version="1" type="test">
  <ConnName>high-side</ConnName>
  <Timestamp>2026-04-12T09:00:00Z</Timestamp>
  <Epoch>epoch01aaaaaaaaaaaaaaaaaaaa</Epoch>
  <TeamName>high-side</TeamName>
  <ChannelName>high-side</ChannelName>
  <TestID>testid12aaaaaaaaaaaaaaaaaaaa</TestID>
</CrossGuardEnvelope>
```


Envelope Types

The `type` attribute is the envelope discriminator. The four constants are defined in `server/transport.go`. Two ride the XML envelope above; two are file uploads with headers instead.

Type Value	Go Constant	Carrier	Description
<code>sync_msg</code>	<code>TransportTypeSyncMsg</code>	XML envelope, <code>SyncMsg</code> payload	Shared-channels sync. Carries at most one Post plus optional non-post content (users, reactions, statuses, membership changes, acknowledgements, mention transforms) from one sync cycle.
<code>test</code>	<code>TransportTypeTest</code>	XML envelope, <code>TestID</code> field	Connectivity check sent by the test-connection handler. Carries <code>Epoch</code> and <code>TestID</code> ; the receiver echoes both into audit logs.
<code>attachment</code>	<code>TransportTypeAttachment</code>	File upload, <code>X-Crossguard-*</code> headers	Raw file bytes for a Post attachment. Not an XML envelope. Metadata travels as HTTP-style headers on the file blob. Emitted from <code>OnSharedChannelsAttachmentSyncMsg</code> .
<code>profile_image</code>	<code>TransportTypeProfileImage</code>	File upload, <code>X-Crossguard-*</code> headers	Raw image bytes for a synced user's avatar. Not an XML envelope. Emitted from <code>OnSharedChannelsProfileImageSyncMsg</code> .

SyncMsg Payload

The `SyncMsg` element carries the shared-channels sync content. The plugin uses pruned, plugin-owned wire types (under `server/wire/`); each is a subset of the corresponding upstream `mmModel` type with source-only fields stripped and receiver-overwritten fields dropped. Field order inside `<SyncMsg>` is fixed: `Id`, `ChannelId`, `Users`, `Post`, `Reactions`, `Statuses`, `MembershipChanges`, `Acknowledgements`, `MentionTransforms`.

Each `sync_msg` envelope carries **at most one** `Post`. The sender splits each upstream sync cycle into one envelope per Post; non-post content from the same cycle (orphan reactions, memberships, statuses, and the users they reference) rides a single optional *metadata envelope*

with no `<Post>` child. This one-post-per-envelope contract is the compliance-content-inspection blast-radius control: rejecting one post does not lose other content as collateral damage.

PAYLOAD **SyncMsg**

Fields

Field	Type	Required	Description
<code>Id</code>	string (26-char Mattermost ID)	Yes	Identifier for the sync cycle (upstream <code>SyncMsg.Id</code>).
<code>ChannelId</code>	string (26-char Mattermost ID)	Yes	Source-side channel ID this sync cycle belongs to.
<code>Users</code>	map (string → User)	No	Users referenced by the contained <code>Post</code> , <code>Reactions</code> , <code>Statuses</code> , <code>MembershipChanges</code> , or <code>Acknowledgements</code> . XML: <code><User id="..."></code> children of <code><Users></code> . JSON: a map keyed by user ID. Sorted by key. Max 1000 entries. Omitted entirely when empty.
<code>Post</code>	Post	No	At most one <code>Post</code> per envelope. Absent on metadata envelopes.
<code>Reactions</code>	array of Reaction	No	Reactions emitted in this sync cycle. Max 1000. Container omitted when empty.
<code>Statuses</code>	array of Status	No	Presence snapshots emitted in this sync cycle. Max 1000. Container omitted when empty.
<code>MembershipChanges</code>	array of MembershipChange	No	Channel join and leave events emitted in this sync cycle. Max 1000. Container omitted when empty.

Field	Type	Required	Description
Acknowledgements	array of PostAcknowledgement	No	Post acknowledgements emitted in this sync cycle. Max 1000. Container omitted when empty.
MentionTransforms	map (string → string)	No	Sender-resolved mention rewrites (e.g. <code>@alice</code> → <code>@alice.high</code>). XML: <code><Transform key="..." value="..."/></code> children of <code><MentionTransforms></code> . JSON: a map. Sorted by key. Max 1000 entries.

Post

Wire-pruned subset of `mmModel.Post`. Source-side computed fields (`ReplyCount`, `LastReplyAt`, `Participants`, `IsFollowing`, `MessageSource`, `PendingPostId`) and receiver-overwritten fields (`FileNames`, `Metadata`) are dropped at the source. Reference Go type: `wire.Post`.

Field	Type	Required	Description
Id	string (26-char Mattermost ID)	Yes	Source-side post ID.
CreateAt	int64	Yes	Unix milliseconds.
UpdateAt	int64	Yes	Unix milliseconds.
EditAt	int64	No	Unix milliseconds. Set on edits.
DeleteAt	int64	Yes	Unix milliseconds. Non-zero on deletes.

Field	Type	Required	Description
UserId	string (26-char Mattermost ID)	Yes	Source-side author ID.
ChannelId	string (26-char Mattermost ID)	Yes	Source-side channel ID.
RootId	string (26-char Mattermost ID)	No	Parent post ID for threaded replies.
OriginalId	string (26-char Mattermost ID)	No	ID of the post this is an edit of. Set on edits.
Message	string (max 1 MiB)	Yes	Markdown-formatted post body.
Type	string (max 96)	No	Empty for normal posts; <code>system_*</code> for system messages (e.g. <code>system_add_to_channel</code>).
Props	PostProps	No	Whitelisted typed Props. See below.
Hashtags	string (max 1536)	No	Space-separated hashtags extracted by the source.
FileIds	array of string (max 10, each 26 chars)	No	IDs of attached files. The file bytes ride a separate <code>attachment</code> upload (see File-Upload Headers), not the XML envelope. XML: <code><FileIds><Id>...</Id>...</FileIds></code> . Container omitted when empty.
HasReactions	bool	No	True if the post had reactions on the source side.
RemoteId	string (26-char Mattermost ID)	No	Source-side remote ID (if the post originated from another shared-channels remote).
IsPinned	bool	No	True if the post is pinned.

PostProps

Whitelisted typed subset of `Post.Props`. Exactly fifteen keys are carried on the wire; any other upstream key is dropped at the boundary. `OverrideIconEmoji` has surrounding colons (`:emoji:`) stripped at marshal and re-added at unmarshal. Reference Go type: `wire.PostProps`.

Field	Type	Description
<code>FromWebhook</code>	bool	Post originated from an incoming webhook.
<code>FromBot</code>	bool	Post originated from a bot account.
<code>FromPlugin</code>	bool	Post originated from a plugin.
<code>FromOAuthApp</code>	bool	Post originated from an OAuth app.
<code>OverrideUsername</code>	string (max 192)	Webhook override for displayed username.
<code>OverrideIconURL</code>	string (max 3072)	Webhook override for displayed icon URL.
<code>OverrideIconEmoji</code>	string (max 96)	Webhook override for displayed icon emoji. Colons stripped on the wire.
<code>WebhookDisplayName</code>	string (max 192)	Webhook-supplied display name.
<code>AddedUserId</code>	string (26 chars)	User added (set on <code>system_add_to_channel</code> system posts).
<code>DeleteBy</code>	string (26 chars)	User who deleted the post.
<code>AddChannelMember</code>	string (26 chars)	Alternate add-member key carried by some upstream paths.
<code>MentionHighlightDisabled</code>	bool	Suppresses @mention highlight for this post.

Field	Type	Description
<code>DisableGroupHighlight</code>	bool	Suppresses group-mention highlight for this post.
<code>AIGeneratedByUserId</code>	string (26 chars)	User ID of the agent that generated this post.
<code>AIGeneratedByUsername</code>	string (max 96)	Username of the agent that generated this post.

User

Wire-pruned subset of `mmModel.User`. Credential fields, notification preferences, and other source-local data are dropped at the boundary. `Username` runs a resolve-then-truncate ladder so colon-tagged remote usernames never reach the wire; the colon-bearing form is recovered from `Props.RemoteUsername` on the receiver when needed. Reference Go type: `wire.User`.

Field	Type	Required	Description
<code>Id</code>	string (26-char Mattermost ID)	Yes	Source-side user ID. XML carries it both as the <code>id</code> attribute on the parent <code><User></code> element and as the <code><Id></code> child for consistency.
<code>CreateAt</code>	int64	No	Unix milliseconds.
<code>UpdatedAt</code>	int64	No	Unix milliseconds.
<code>DeleteAt</code>	int64	Yes	Unix milliseconds. Non-zero for tombstones.
<code>Username</code>	string (max 96)	Yes	Lowercase alnum + <code>-_.</code> . Colon-tagged remote forms are stripped.
<code>Email</code>	string (max 381)	Yes	RFC-style email address.

Field	Type	Required	Description
Nickname	string (max 192)	No	User-chosen nickname.
FirstName	string (max 192)	No	First name.
LastName	string (max 192)	No	Last name.
Position	string (max 1536)	No	Free-form job-title field.
Roles	string (max 1536)	Yes	Space-separated role names (e.g. <code>system_user</code> <code>system_admin</code>).
Locale	string (max 53)	No	BCP 47-shaped locale code.
Timezone	map (string → string)	No	Open string map (max 100 entries). XML: <code><Entry key="..." value="..." /></code> children of <code><Timezone></code> . JSON: a flat map. Sorted by key. Omitted when empty.
Props	UserProps	No	Whitelisted typed Props. See below.
RemoteId	string (26 chars)	No	Source-side remote ID (if the user originated from another shared-channels remote).
IsBot	bool	No	True for bot accounts.
BotDescription	string (max 1536)	No	Description of the bot account.
BotLastIconUpdate	int64	No	Unix milliseconds when the bot icon was last updated.

UserProps

Whitelisted subset of `User.Props` . Pattern-validated values (`RemoteUsername` , `RemoteEmail`) that fail validation are dropped at marshal with a Warn-level audit log; the user record itself is kept. Reference Go type: `wire.UserProps` .

Field	Type	Description
<code>CustomStatus</code>	string (max 1536)	Custom-status JSON blob (upstream wire shape preserved verbatim).
<code>RemoteUsername</code>	string (max 96)	Original colon-tagged form of the username (e.g. <code>alice.high</code>) when the wire <code>Username</code> field has been resolved to a plain slug.
<code>RemoteEmail</code>	string (max 381)	Original email address from a previous remote.
<code>OriginalRemoteId</code>	string (26 chars)	The remote ID this user originated from before passing through intermediate relays.

Reaction

An emoji reaction on a post. Reference Go type: `wire.Reaction` .

Field	Type	Required	Description
<code>UserId</code>	string (26 chars)	Yes	User who reacted.
<code>PostId</code>	string (26 chars)	Yes	Post the reaction is on.
<code>EmojiName</code>	string (max 96)	Yes	Emoji name without surrounding colons (e.g. <code>thumbsup</code>).
<code>CreateAt</code>	int64	Yes	Unix milliseconds.
<code>UpdatedAt</code>	int64	Yes	Unix milliseconds.

Field	Type	Required	Description
DeleteAt	int64	Yes	Unix milliseconds. Non-zero on reaction removals.
RemoteId	string (26 chars)	No	Source-side remote ID (if the reaction originated from another remote).
ChannelId	string (26 chars)	Yes	Channel the reacted-to post lives in. Back-filled from the parent <code>SyncMsg.ChannelId</code> when upstream omits it.

Status

Presence snapshot. Source-local `ActiveChannel` is dropped at the boundary. Reference Go type: `wire.Status`.

Field	Type	Required	Description
UserId	string (26 chars)	Yes	User whose presence is being reported.
Status	enum: <code>online</code> <code>away</code> <code>dnd</code> <code>offline</code>	Yes	Current presence.
Manual	bool	Yes	True if the user manually set this status (rather than it being automatic).
LastActivityAt	int64	Yes	Unix milliseconds.
DNDEndTime	int64	Yes	Unix seconds (upstream quirk preserved on the wire). When the user's Do-Not-Disturb status expires.

MembershipChange

A channel join or leave event. Reference Go type: `wire.MembershipChange`.

Field	Type	Required	Description
<code>ChannelId</code>	string (26 chars)	Yes	Channel the user joined or left.
<code>UserId</code>	string (26 chars)	Yes	User whose membership changed.
<code>IsAdd</code>	bool	Yes	True for a join, false for a leave.
<code>RemoteId</code>	string (26 chars)	No	Source-side remote ID (if the change originated from another remote).
<code>ChangeTime</code>	int64	Yes	Unix milliseconds.

PostAcknowledgement

A user's acknowledgement of a post. Reference Go type: `wire.PostAcknowledgement`.

Field	Type	Required	Description
<code>UserId</code>	string (26 chars)	Yes	User who acknowledged.
<code>PostId</code>	string (26 chars)	Yes	Post that was acknowledged.
<code>AcknowledgedAt</code>	int64	Yes	Unix milliseconds.
<code>ChannelId</code>	string (26 chars)	Yes	Channel the post lives in.
<code>RemoteId</code>	string (26 chars)	No	Source-side remote ID (if the acknowledgement originated from another remote).

File-Upload Headers

`attachment` and `profile_image` envelopes do not ride the XML transport envelope. The plugin uploads raw file bytes through `QueueProvider.UploadFile` ; metadata travels as HTTP-style headers on the file blob. Each provider maps these headers to its native metadata mechanism: NATS JetStream Object Store uses `nats.Header` , Azure Blob uses blob metadata, Azure Service Bus uses metadata on the blob sidecar container. The receiver reads them on the `WatchFiles` event and reconstructs the file context. Header names are case-insensitive on the wire.

Header	Required For	Description
<code>X-Crossguard-Kind</code>	both	Discriminator: <code>attachment</code> or <code>profile_image</code> .
<code>X-Conn-Name</code>	both	Outbound connection name on the sender (URL-safe slug, max 96 chars).
<code>X-Filename</code>	both	Original filename. <code>profile.png</code> for profile images.
<code>X-Team-Name</code>	<code>attachment</code>	Sender team name. The receiver runs the same rewrite-then-by-name resolution the message envelope uses.
<code>X-Channel-Name</code>	<code>attachment</code>	Sender channel name. Same rewrite path.
<code>X-Post-Id</code>	<code>attachment</code>	26-character Mattermost ID of the post this file belongs to.
<code>X-File-Info</code>	<code>attachment</code>	Base64-encoded JSON <code>mmModel.FileInfo</code> . Base64 wraps the JSON so it survives provider metadata-encoding rules (Azure Blob metadata is HTTP-header-grade; NATS uses <code>nats.Header</code>).
<code>X-User-Id</code>	<code>profile_image</code>	26-character Mattermost ID of the synced user whose avatar is being uploaded.

Object keys follow a stable pattern: `attachment/{postID}/{fileID}` for attachments and `profile_image/{userID}` for profile images. The receiver uses the key prefix and the `x-`

`Crossguard-Kind` header in tandem so mistyped uploads cannot escape their lane.

Complete Wire Format Examples

The repository ships 22 schema-valid XML fixtures under `schema/examples/` that span the full range of envelope content (simple posts, edits, deletes, reactions, threaded replies, system messages, membership changes, status snapshots, acknowledgements, mention transforms, typed Props, bot users, metadata envelopes, and a connectivity test). Each fixture is byte-for-byte identical to the bytes the plugin emits on the wire. See `schema/examples/README.md` for the index. The examples below show the XML wire form indented for readability, followed by the JSON equivalent. Field names mirror exactly across the two serializations.

Sync Envelope (one post, one user)

This is the canonical case: Alice posts in `#general` on team A. The envelope carries the post plus the author's user record inline. Source fixture: `01_post_simple.xml`.

XML

```
<?xml version="1.0" encoding="UTF-8"?>
<CrossGuardEnvelope version="1" type="sync_msg">
  <ConnName>nats-low-to-high</ConnName>
  <Timestamp>2026-05-11T10:00:00Z</Timestamp>
  <Epoch>epoch01aaaaaaaaaaaaaaaaaaaa</Epoch>
  <Sequence>1</Sequence>
  <TeamName>team-a</TeamName>
  <ChannelName>general</ChannelName>
  <SyncMsg>
    <Id>sm01aaaaaaaaaaaaaaaaaaaaaaaa</Id>
    <ChannelId>ch0aaaaaaaaaaaaaaaaaaaaaaaa</ChannelId>
    <Users>
      <User id="u01aliceaaaaaaaaaaaaaaaaaaaa">
        <Id>u01aliceaaaaaaaaaaaaaaaaaaaa</Id>
        <CreateAt>1700000000000</CreateAt>
        <UpdateAt>1712956800000</UpdateAt>
        <DeleteAt>0</DeleteAt>
        <Username>alice</Username>
        <Email>alice@server-a.example.com</Email>
        <Nickname>Alice</Nickname>
        <FirstName>Alice</FirstName>
```

```

        <LastName>Smith</LastName>
        <Roles>system_user</Roles>
        <Locale>en</Locale>
        <RemoteId>remoteloooooooooooooooo</RemoteId>
    </User>
</Users>
<Post>
    <Id>p01postaaaaaaaaaaaaaaaa</Id>
    <CreateAt>1712956800000</CreateAt>
    <UpdateAt>1712956800000</UpdateAt>
    <DeleteAt>0</DeleteAt>
    <UserId>u01aliceaaaaaaaaaaaaaaaa</UserId>
    <ChannelId>ch0aaaaaaaaaaaaaaaa</ChannelId>
    <Message>Morning team. Standup at 9:30 in the usual room, see you there.</Mes
sage>
</Post>
</SyncMsg>
</CrossGuardEnvelope>

```

JSON

```

{
  "version": 1,
  "type": "sync_msg",
  "ConnName": "nats-low-to-high",
  "Timestamp": "2026-05-11T10:00:00Z",
  "Epoch": "epoch01aaaaaaaaaaaaaaaa",
  "Sequence": 1,
  "TeamName": "team-a",
  "ChannelName": "general",
  "SyncMsg": {
    "Id": "sm01aaaaaaaaaaaaaaaa",
    "ChannelId": "ch0aaaaaaaaaaaaaaaa",
    "Users": {
      "u01aliceaaaaaaaaaaaaaaaa": {
        "Id": "u01aliceaaaaaaaaaaaaaaaa",
        "CreateAt": 1700000000000,
        "UpdateAt": 1712956800000,
        "DeleteAt": 0,
        "Username": "alice",
        "Email": "alice@server-a.example.com",
        "Nickname": "Alice",
        "FirstName": "Alice",
        "LastName": "Smith",
        "Roles": "system_user",

```

```

        "Locale": "en",
        "RemoteId": "remoteloooooooooooooooooooo"
    }
},
"Post": {
    "Id": "p01postaaaaaaaaaaaaaaaaaaaaa",
    "CreateAt": 1712956800000,
    "UpdateAt": 1712956800000,
    "DeleteAt": 0,
    "UserId": "u01aliceaaaaaaaaaaaaaaaaaaaaa",
    "ChannelId": "ch0aaaaaaaaaaaaaaaaaaaaa",
    "Message": "Morning team. Standup at 9:30 in the usual room, see you there."
}
}
}

```

Sync Envelope (post delete)

Deletes are signalled by a non-zero `DeleteAt` with an empty `Message` and a `Props.DeleteBy` carrying the deleting user. Source fixture: `08_delete.xml`.

```

<?xml version="1.0" encoding="UTF-8"?>
<CrossGuardEnvelope version="1" type="sync_msg">
  <ConnName>nats-low-to-high</ConnName>
  <Timestamp>2026-05-11T10:00:08Z</Timestamp>
  <Epoch>epoch01aaaaaaaaaaaaaaaaaaaaa</Epoch>
  <Sequence>8</Sequence>
  <TeamName>team-a</TeamName>
  <ChannelName>general</ChannelName>
  <SyncMsg>
    <Id>sm08aaaaaaaaaaaaaaaaaaaaa</Id>
    <ChannelId>ch0aaaaaaaaaaaaaaaaaaaaa</ChannelId>
    <Post>
      <Id>p01postaaaaaaaaaaaaaaaaaaaaa</Id>
      <CreateAt>1712956800000</CreateAt>
      <UpdateAt>1712956880000</UpdateAt>
      <DeleteAt>1712956880000</DeleteAt>
      <UserId>u01aliceaaaaaaaaaaaaaaaaaaaaa</UserId>
      <ChannelId>ch0aaaaaaaaaaaaaaaaaaaaa</ChannelId>
      <Message></Message>
      <Props><DeleteBy>u01aliceaaaaaaaaaaaaaaaaaaaaa</DeleteBy></Props>
    </Post>
  </SyncMsg>
</CrossGuardEnvelope>

```

```
</SyncMsg>
</CrossGuardEnvelope>
```

Metadata Envelope (no `<Post>` , one orphan reaction)

Metadata envelopes carry only non-post content from a sync cycle: orphan reactions and acks on posts from previous cycles, plus the users referenced by them. The `<Post>` element is absent. Source fixture: `21_metadata_orphan_reaction.xml` .

```
<?xml version="1.0" encoding="UTF-8"?>
<CrossGuardEnvelope version="1" type="sync_msg">
  <ConnName>nats-low-to-high</ConnName>
  <Timestamp>2026-05-11T10:00:00Z</Timestamp>
  <Epoch>epoch21aaaaaaaaaaaaaaaaaaaa</Epoch>
  <Sequence>1</Sequence>
  <TeamName>team-a</TeamName>
  <ChannelName>general</ChannelName>
  <SyncMsg>
    <Id>sm21aaaaaaaaaaaaaaaaaaaaaaaa</Id>
    <ChannelId>ch0aaaaaaaaaaaaaaaaaaaa</ChannelId>
    <Users>...</Users>
    <Reactions>
      <Reaction>
        <UserId>u01aliceaaaaaaaaaaaaaaaa</UserId>
        <PostId>p99orphanaaaaaaaaaaaaaaaa</PostId>
        <EmojiName>thumbsup</EmojiName>
        <CreateAt>1712957000000</CreateAt>
        <UpdateAt>1712957000000</UpdateAt>
        <DeleteAt>0</DeleteAt>
        <ChannelId>ch0aaaaaaaaaaaaaaaaaaaa</ChannelId>
      </Reaction>
    </Reactions>
  </SyncMsg>
</CrossGuardEnvelope>
```

Connectivity Test

Test envelopes carry `Epoch` and `TestID` but no `Sequence` (no channel scope). `TeamName` and `ChannelName` are stamped with the connection name as a placeholder so the envelope passes XSD validation; the receiver ignores both fields for `type=test` . Source fixture: `12_test.xml` .

XML

```
<?xml version="1.0" encoding="UTF-8"?>
<CrossGuardEnvelope version="1" type="test">
  <ConnName>nats-low-to-high</ConnName>
  <Timestamp>2026-05-11T10:00:00Z</Timestamp>
  <Epoch>epoch01aaaaaaaaaaaaaaaaaaaa</Epoch>
  <TeamName>nats-low-to-high</TeamName>
  <ChannelName>nats-low-to-high</ChannelName>
  <TestID>testid12aaaaaaaaaaaaaaaaaaaa</TestID>
</CrossGuardEnvelope>
```

JSON

```
{
  "version": 1,
  "type": "test",
  "ConnName": "nats-low-to-high",
  "Timestamp": "2026-05-11T10:00:00Z",
  "Epoch": "epoch01aaaaaaaaaaaaaaaaaaaa",
  "TeamName": "nats-low-to-high",
  "ChannelName": "nats-low-to-high",
  "TestID": "testid12aaaaaaaaaaaaaaaaaaaa"
}
```

Connection Configuration (NATS)

Each NATS connection (inbound or outbound) is configured in the System Console. This schema is stored in plugin settings and is not transmitted over the wire. It is included here for reference when setting up NATS integrations. The `provider` field selects the transport backend. NATS-specific fields (`address`, `subject`, `tls_enabled`, etc.) are nested under the `"nats"` key in JSON.

CONFIG NATSConnection

Fields

Field	Type	Required	Description
-------	------	----------	-------------

Connection Fields (top-level in JSON)			
<code>name</code>	string	Yes	Connection identifier. Must start and end with a lowercase letter or number. Hyphens allowed between segments, no consecutive hyphens (pattern: <code>^[a-z0-9]+(-[a-z0-9]+)*\$</code>).
<code>provider</code>	string	No	Transport provider: <code>"nats"</code> , <code>"azure-queue"</code> , <code>"azure-blob"</code> , or <code>"azure-servicebus"</code> . Default: <code>"nats"</code> .
<code>file_transfer_enabled</code>	boolean	No	Enable file attachment relay via JetStream Object Store. Default: <code>false</code> .
<code>file_filter_mode</code>	string	No	File extension filter: <code>""</code> (no filter), <code>"allow"</code> (whitelist), or <code>"deny"</code> (blacklist). Default: <code>""</code> .
<code>file_filter_types</code>	string	Conditional	Comma-separated extensions (e.g. <code>.pdf, .docx, .png</code>). Case-insensitive. Required when <code>file_filter_mode</code> is <code>"allow"</code> or <code>"deny"</code> .
<code>message_format</code>	string	No	Wire format for outbound messages: <code>"json"</code> (default) or <code>"xml"</code> . Use XML when sending through a Cross Domain Solution (CDS). Inbound messages are auto-detected regardless of this setting.

NATS Provider Fields (nested under <code>"nats"</code> key in JSON)			
<code>addresses</code>	string	Yes	NATS server address (e.g. <code>nats://nats:4222</code>).
<code>subject</code>	string	Yes	NATS subject. Must start with <code>crossguard.</code> (e.g. <code>crossguard.high</code>).
<code>tls_enabled</code>	boolean	No	Enable TLS encryption. Default: <code>false</code> .

Field	Type	Required	Description
<code>auth_type</code>	string	No	Authentication method: <code>none</code> , <code>token</code> , or <code>credentials</code> . Default: <code>none</code> .
<code>token</code>	string	Conditional	Bearer token. Required when <code>auth_type</code> is <code>token</code> .
<code>username</code>	string	Conditional	Username. Required when <code>auth_type</code> is <code>credentials</code> .
<code>password</code>	string	Conditional	Password. Required when <code>auth_type</code> is <code>credentials</code> .
<code>client_certificate</code>	string	No	Filesystem path to the client TLS certificate.
<code>client_key</code>	string	No	Filesystem path to the client TLS private key.
<code>ca_certificate</code>	string	No	Filesystem path to the CA certificate for server verification.

Example: Token Authentication

```
{
  "name": "high-side",
  "provider": "nats",
  "nats": {
    "address": "nats://nats:4222",
    "subject": "crossguard.high",
    "tls_enabled": false,
    "auth_type": "token",
    "token": "my-secret-token"
  }
}
```

Example: TLS with Credentials

```
{
  "name": "secure-relay",
  "provider": "nats",
```

```
"nats": {
  "address": "nats://nats.example.com:4222",
  "subject": "crossguard.relay",
  "tls_enabled": true,
  "auth_type": "credentials",
  "username": "crossguard-svc",
  "password": "s3cret",
  "client_cert": "/etc/crossguard/client.crt",
  "client_key": "/etc/crossguard/client.key",
  "ca_cert": "/etc/crossguard/ca.crt"
}
}
```

Example: File Transfer with Allow Filter

```
{
  "name": "high-side",
  "provider": "nats",
  "file_transfer_enabled": true,
  "file_filter_mode": "allow",
  "file_filter_types": ".pdf,.docx,.png,.jpg",
  "nats": {
    "address": "nats://nats:4222",
    "subject": "crossguard.high",
    "auth_type": "token",
    "token": "my-secret-token"
  }
}
```

Example: XML Format for CDS

```
{
  "name": "cds-relay",
  "provider": "nats",
  "message_format": "xml",
  "nats": {
    "address": "nats://nats:4222",
    "subject": "crossguard.cds",
    "tls_enabled": true,
    "auth_type": "token",
    "token": "my-secret-token"
  }
}
```

```
}  
}
```

Connection Behavior (NATS)

The plugin uses the standard NATS client (`nats.go`) with its default reconnect machinery. The shared-channels framework owns the retry semantics for sync envelopes: a failed `OnSharedChannelsSyncMsg` call returns an error and leaves the framework cursor unadvanced, so the framework retries on its own schedule. The plugin does not add a publish-side retry loop. Inbound work is deliberately serial under the single-active-receiver lease, so there is no relay semaphore.

Parameter	Value	Description
Connect Timeout (test)	30 seconds	One-time connections used by the test-connection API endpoint.
Connect Timeout (relay)	5 seconds	Persistent relay connections established at plugin activation.
Max Reconnects	Unlimited	Relay connections attempt reconnection indefinitely.
Reconnect Wait	2 seconds	Delay between reconnection attempts.
Object Store TTL	1 hour	JetStream Object Store bucket TTL for file objects.
Inbound Active Lease	45 s TTL, 15 s renewal	KV TTL lease held by the active inbound receiver. Only one node in the cluster subscribes per inbound connection at a time; failover is automatic.

File Transfer Protocol (NATS JetStream)

File attachment relay uses NATS JetStream Object Store as the `QueueProvider.UploadFile / WatchFiles` backend. Sync envelopes (NATS core pub/sub) and file uploads (JetStream) share the same NATS connection but ride independent NATS subjects, so a backlog on one does not block the other. Profile-image uploads use the same Object Store with a different key prefix.

Object Store Configuration

Parameter	Value	Description
Bucket Name	<code>crossguard-files</code>	Auto-created on first use via <code>CreateOrUpdateObjectStore</code> .
TTL	1 hour	Objects are automatically purged after TTL. No receiver-side deletion (supports fan-out to multiple receivers).
Object Key (attachment)	<code>attachment/{postID}/{fileID}</code>	Source-side post ID plus the source-side file ID.
Object Key (profile image)	<code>profile_image/{userID}</code>	Source-side user ID. Avatars are overwritten on update.
Max File Size	Server's <code>MaxFileSize</code>	Governed by <code>FileSettings.MaxFileSize</code> in the Mattermost server config. Default 100 MB.

Object Metadata Headers

Each uploaded object carries a set of custom NATS headers that the receiving instance uses to route the file to the correct context. The header set is the same across all four transports (the headers are [documented in detail above](#)): the discriminator `X-Crossguard-Kind` (`attachment` or `profile_image`) plus the kind-specific routing headers (`X-Post-Id` , `X-Team-Name` , `X-Channel-Name` , `X-File-Info` for attachments; `X-User-Id` for profile images). On the inbound side, the handler reads these headers from the `WatchFiles` event and runs the same rewrite-then-by-name resolution the sync envelope uses.

Outbound File Flow

User posts a message with a file attachment (or updates an avatar)
|

```

    v
Shared-channels framework fires OnSharedChannelsAttachmentSyncMsg
(or OnSharedChannelsProfileImageSyncMsg for avatars)
    |
    v
Resolve outbound connection via connNameForRemote(rc.RemoteId)
    |
    v
Check: file_transfer_enabled? File passes filter? Under size limit?
    |
    v
Read file bytes from Mattermost (API.GetFile / GetProfileImage)
    |
    v
Get or create JetStream Object Store bucket ("crossguard-files")
    |
    v
QueueProvider.UploadFile:
    key:      attachment/{postID}/{fileID}    (or profile_image/{userID})
    headers: X-Crossguard-Kind, X-Conn-Name, X-Filename,
             X-Team-Name, X-Channel-Name, X-Post-Id, X-File-Info
             (or X-Crossguard-Kind, X-Conn-Name, X-Filename, X-User-Id
              for profile images)

```

Inbound File Flow

```

Active receiver subscribes to the Object Store via QueueProvider.WatchFiles
(single-active election, same KV TTL lease that gates sync subscribe)
    |
    v
JetStream delivers a new object (UpdatesOnly mode)
    |
    v
Read X-Crossguard-Kind header to discriminate attachment / profile_image
    |
    v
Filter: does X-Conn-Name match this inbound connection?
        does the file pass the inbound filter policy?
    |
    v
Resolve target team and channel via rewrite-then-by-name
(same path the sync envelope uses)
    |
    v
Hand to the shared-channels framework:

```

```
* attachment: ReceiveSharedChannelAttachment with X-File-Info metadata
* profile_image: ReceiveSharedChannelProfileImage with X-User-Id
|
v
```

Framework performs the local upload, attachment-to-post wiring, or avatar update; the plugin does not maintain a separate post-ID mapping.

File Filtering

Each connection can independently filter files by extension using three modes:

Mode	Behavior	Example
<code>""</code> (empty, default)	No filtering. All files pass.	<code>"file_filter_mode": ""</code>
<code>"allow"</code>	Only files with listed extensions are relayed.	<code>"file_filter_mode": "allow", "file_filter_types": ".pdf, .docx"</code>
<code>"deny"</code>	Files with listed extensions are blocked.	<code>"file_filter_mode": "deny", "file_filter_types": ".exe, .bat"</code>

Extensions are case-insensitive and a leading dot is optional (both `pdf` and `.pdf` are accepted). Filters are evaluated independently on outbound (before upload) and inbound (before download).

JetStream Requirement

File transfer requires a JetStream-enabled NATS server. If the NATS server does not have JetStream enabled, file transfer will fail silently with errors logged. Core NATS text relay is not affected.

Format Selection: XML and JSON

Cross Guard supports two wire serializations. The outbound connection's `message_format` setting (`xml` or `json`) selects which one outbound envelopes use. Inbound connections are auto-detected; a single inbound connection can accept envelopes in either format. JSON field names exactly mirror the XML element / attribute names defined in `schema/crossguard.xsd`, so the two serializations are field-for-field equivalent.

Key Differences Between Formats

Aspect	XML	JSON
Root	<code><CrossGuardEnvelope version="1" type="..."></code> (no namespace; <code>version</code> and <code>type</code> are attributes)	Top-level object with <code>version</code> and <code>type</code> fields
Field names	CamelCase elements (<code><ConnName></code> , <code><Timestamp></code> , <code><SyncMsg></code>); lowercase root attributes (<code>version</code> , <code>type</code>)	Same names, same casing (mirror)
Character escaping	XML entity escaping (<code>&lt;</code> , <code>&gt;</code> , <code>&amp;</code> , <code>&#xA;</code> for newlines)	JSON string escaping
Schema	<code>schema/crossguard.xsd</code> is the normative XML Schema; every outbound payload is schema-valid by construction	OpenAPI 3.1 schema in <code>schema/crossguard-api.yaml</code>
Empty containers	Omitted entirely (no empty <code><Users></Users></code>)	Omitted entirely (no <code>"Users": {}</code>)
Map ordering	Sorted by key (XML <code><Entry></code> / <code><Transform></code> children)	Sorted by key (deterministic JSON object property order)
Size overhead	Baseline (closing tags)	10-20% smaller in typical envelopes

Inbound Auto-Detection

Inbound connections do not require format configuration. The plugin inspects the first non-whitespace byte of each incoming envelope: `<` selects the XML parser, `{` selects the JSON parser. A single inbound connection can therefore accept envelopes from sources that use either format.

BOM Handling

XML output does not include a Byte Order Mark. If an upstream device (e.g. a CDS shuttle) prepends a UTF-8 BOM (`0xEF 0xBB 0xBF`) to the message, the inbound parser strips it before format detection so BOM-prefixed XML still parses correctly. JSON envelopes do not carry a BOM.

Schema-Valid Example Payloads

The repository ships 22 schema-valid XML fixtures under `schema/examples/`. They collectively exercise simple posts, markdown-rich posts, fenced code blocks with literal `<`, `>`, and `&` in the body, GFM tables and reference links, threaded replies, long-form incident reports, post edits, deletes, standard-emoji and custom-emoji reactions, reaction removal, the connectivity test envelope, channel join and leave events, presence statuses with DND, post acknowledgements, sorted mention transforms, typed Props (webhook and AI provenance), users with Timezone maps, bot users, metadata envelopes (orphan content), and system messages with typed system Props. The `README` walks through them.

Each fixture is **byte-for-byte identical** to the bytes the plugin puts on the wire: the output of `MarshalEnvelope` plus a trailing newline. That means a single line per envelope after the `<?xml?>` header (no inter-element indentation), entity-escaped string content, and no CDATA sections. The byte-equality contract is enforced by `TestExampleFiles` in `server/examples_roundtrip_test.go`, which also verifies that `UnmarshalEnvelope` followed by `MarshalEnvelope` is a fixed point and that marshalling is deterministic.

All sample IDs are 26 characters of `[a-z0-9]` (per the `MattermostIdType` constraint in `schema/crossguard.xsd`). Envelope `Timestamp` is ISO 8601 (RFC 3339); content timestamps inside `SyncMsg` are Unix epoch milliseconds (`Status.DNDEndTime` is the upstream-quirk exception, in seconds).

To validate the fixtures locally:

```
for f in schema/examples/*.xml; do
  xllint --noout --schema schema/crossguard.xsd "$f"
done
```

The integration suite has an opt-in mode (`make docker-integration-test-validate-wire`) that archives every envelope the running plugin emits and validates each one against `crossguard.xsd` the same way. See [the README](#) for the wiring.

The simplest fixture (a minimal top-level post) is reproduced below in raw wire form. Open the file in an editor with soft-wrap, or pipe through `xllint --format`, for a human-readable view.

Example: `01_post_simple.xml` (raw wire form)

```
<?xml version="1.0" encoding="UTF-8"?>
<CrossGuardEnvelope version="1" type="sync_msg"><ConnName>nats-low-to-high</ConnName>
<Timestamp>2026-05-11T10:00:00Z</Timestamp><Epoch>epoch01aaaaaaaaaaaaaaaaaaaa</Epoch>
<Sequence>1</Sequence><TeamName>team-a</TeamName><ChannelName>general</ChannelName>
<SyncMsg><Id>sm01aaaaaaaaaaaaaaaaaaaa</Id><ChannelId>ch0aaaaaaaaaaaaaaaaaaaa</ChannelId>
<Users><User id="u01aliceaaaaaaaaaaaaaaaa"><Id>u01aliceaaaaaaaaaaaaaaaa</Id>
<CreateAt>1700000000000</CreateAt><UpdateAt>1712956800000</UpdateAt><DeleteAt>0</DeleteAt>
<Username>alice</Username><Email>alice@server-a.example.com</Email>
<Nickname>Alice</Nickname><FirstName>Alice</FirstName><LastName>Smith</LastName><Roles>system_user</Roles>
<Locale>en</Locale><RemoteId>remotel0aaaaaaaaaaaaaaaaaaaa</RemoteId></User></Users>
<Post><Id>p01postaaaaaaaaaaaaaaaaaaaa</Id><CreateAt>1712956800000</CreateAt>
<UpdateAt>1712956800000</UpdateAt><DeleteAt>0</DeleteAt><UserId>u01aliceaaaaaaaaaaaaaaaa</UserId>
<ChannelId>ch0aaaaaaaaaaaaaaaaaaaa</ChannelId><Message>Morning team. Standup at 9:30 in the usual room, see you there.</Message></Post>
</SyncMsg></CrossGuardEnvelope>
```

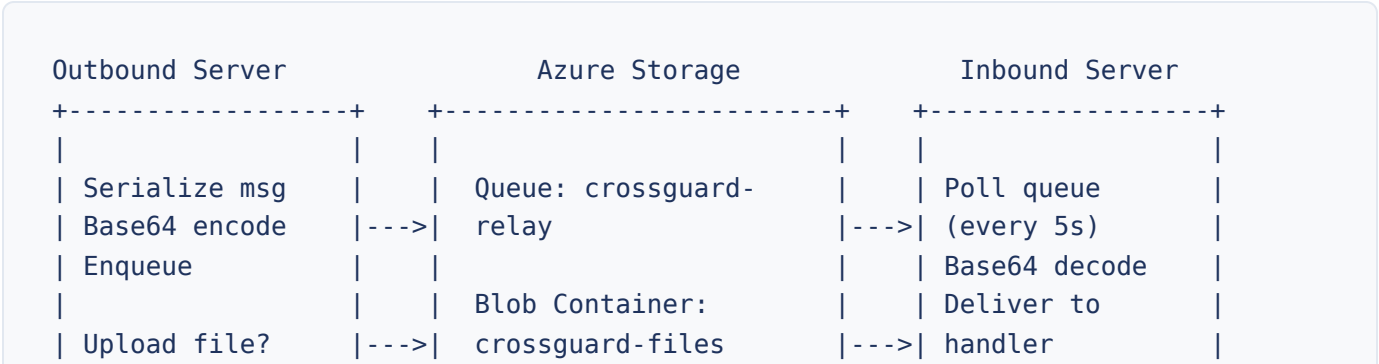
Azure Queue Storage Transport

Overview

The Azure provider uses Azure Queue Storage for message delivery and Azure Blob Storage for file transfer. Unlike NATS pub/sub, Azure uses a polling model where the inbound side periodically checks the queue for new messages.

Auth is via an Azure Storage shared-key credential (legacy default) or an Azure AD Service Principal (Phase 1 supports the client-credentials / client-secret flow). See [Azure Service Principal Authentication](#) in the admin guide for the SP field reference, the minimum RBAC roles, and the deployment guidance (including the no-auto-create rule in SP mode).

Message Flow



```
| Set metadata | | (headers in metadata) | | (every 15s poll) |
+-----+ +-----+ +-----+
```

Queue Polling

Parameter	Value	Description
Poll Interval	5 seconds	How often the inbound side checks for new queue messages
Visibility Timeout	5 minutes	How long a dequeued message is hidden from other consumers
Batch Size	32 messages	Maximum messages dequeued per poll cycle
Message Encoding	Base64	All messages are Base64-encoded before enqueueing
Max Message Size	48,000 bytes	Safe limit before Base64 encoding to the 64 KB Azure wire limit

Message Lifecycle

1. Outbound serializes the message envelope (JSON or XML)
2. Payload is Base64-encoded and enqueued to Azure Queue Storage
3. Inbound polls the queue every 5 seconds
4. Messages are dequeued with a 5-minute visibility timeout
5. On successful processing, the message is deleted from the queue
6. On handler error, the message becomes visible again after the timeout for retry
7. Malformed messages (invalid Base64) are deleted immediately to prevent reprocessing loops

Azure File Transfer (Blob Storage)

When `file_transfer_enabled` is `true` and a `blob_container_name` is configured, file attachments are transferred via Azure Blob Storage.

Upload (Outbound)

1. File data is uploaded as a block blob to the configured container
2. The blob key follows the format: `{post_id}/{unique_id}`

3. Headers (post ID, connection name, filename) are stored as blob metadata under the

`crossguard_headers` key

4. Metadata is a Base64-encoded JSON object: `{"headers": {"X-Post-Id": "...", "X-Conn-Name": "...", "X-Filename": "..."} }`

Download (Inbound)

1. The blob watcher polls the container every 15 seconds using a list operation
2. New blobs (after the last processed marker) are downloaded
3. Headers are extracted from the `crossguard_headers` metadata
4. After successful processing, the blob is deleted
5. The marker only advances after a successful delete, ensuring blobs are re-listed on failure

Azure Queue Connection Configuration

Field	JSON Key	Required	Description
Queue Service URL	<code>queue_service_url</code>	Yes	Azure Queue Storage service URL (e.g. <code>https://<account>.queue.core.windows.net</code>). For US Government, use <code>*.queue.core.usgovcloudapi.net</code> .
Blob Service URL	<code>blob_service_url</code>	When file transfer enabled	Azure Blob Storage service URL (e.g. <code>https://<account>.blob.core.windows.net</code>)
Auth Mode	<code>auth_mode</code>	No	Either <code>shared-key</code> (the legacy default; empty string is treated as the same) or <code>service-principal</code> . Selects which credential fields below apply.
Azure Cloud	<code>azure_cloud</code>	No	Either <code>public</code> (default), <code>usgov</code> , or <code>china</code> . Selects the Azure AD authority and (for Service Principal) the data-plane SDK options. Ignored in shared-key mode (the storage URL alone disambiguates).
Account Name	<code>account_name</code>	Shared-key only	Azure Storage account name. Required when <code>auth_mode</code> is <code>shared-key</code> .

Field	JSON Key	Required	Description
Account Key	<code>account_key</code>	Shared-key only	Azure Storage shared access key (base64, treat as secret). Required when <code>auth_mode</code> is <code>shared-key</code> . MUST be empty when <code>auth_mode</code> is <code>service-principal</code> .
Tenant ID	<code>tenant_id</code>	SP only	Azure AD tenant ID. Required when <code>auth_mode</code> is <code>service-principal</code> .
Client ID	<code>client_id</code>	SP only	Azure AD application (client) ID. Required when <code>auth_mode</code> is <code>service-principal</code> .
Client Secret	<code>client_secret</code>	SP only	Azure AD client secret value (treat as secret). Required when <code>auth_mode</code> is <code>service-principal</code> . Best stored via the <code>MM_PLUGINSETTINGS_PLUGINS_CROSSGUARD_OUTBOUND_CONNECTIONS</code> environment variable rather than the plugin config file.
Queue Name	<code>queue_name</code>	Yes	Azure Queue name. Auto-created in shared-key mode; must be pre-provisioned in Service Principal mode.
Blob Container Name	<code>blob_container_name</code>	When file transfer enabled	Azure Blob container. Auto-created in shared-key mode; must be pre-provisioned in Service Principal mode.
Poll Interval	<code>poll_interval_seconds</code>	No	Inbound queue poll cadence. Default 5, minimum 1.
Blob Poll Interval	<code>blob_poll_interval_seconds</code>	No	Inbound blob-container poll cadence (for file attachments). Default 15, minimum 1.

Example: Azure Queue (shared-key)

```
{
  "name": "azure-relay",
```

```

"provider": "azure-queue",
"file_transfer_enabled": true,
"azure_queue": {
  "queue_service_url": "https://mystorageaccount.queue.core.windows.net",
  "blob_service_url": "https://mystorageaccount.blob.core.windows.net",
  "account_name": "mystorageaccount",
  "account_key": "base64-account-key",
  "queue_name": "crossguard-relay",
  "blob_container_name": "crossguard-files"
}
}

```

Example: Azure Queue (Service Principal)

```

{
  "name": "azure-relay",
  "provider": "azure-queue",
  "file_transfer_enabled": true,
  "azure_queue": {
    "queue_service_url": "https://mystorageaccount.queue.core.windows.net",
    "blob_service_url": "https://mystorageaccount.blob.core.windows.net",
    "auth_mode": "service-principal",
    "azure_cloud": "public",
    "tenant_id": "000000000-0000-0000-0000-000000000000",
    "client_id": "11111111-1111-1111-1111-111111111111",
    "client_secret": "...",
    "queue_name": "crossguard-relay",
    "blob_container_name": "crossguard-files"
  }
}

```

Auto-Provisioning

In `shared-key` mode the Azure Queue provider automatically creates both the queue and the blob container on startup if they do not already exist (the "already exists" response is treated as success). In `service-principal` mode, auto-provisioning is **skipped**: operators must pre-create the queue and (when file transfer is enabled) the blob container so the SP can run with data-plane-only RBAC (typically *Storage Queue Data Contributor* and *Storage Blob Data Contributor*).

Azure Blob Storage Transport

Overview

The Azure Blob provider is a second Azure transport that does not use Azure Queue Storage at all. Instead of enqueueing one queue message per relay event, the outbound side appends messages to an in-memory write-ahead log (WAL) and periodically seals and uploads the WAL as a single batch blob. The inbound side polls the blob container, takes a short-lived lease on each batch blob, processes its messages, and deletes the blob. File attachments are uploaded as deferred file blobs into the same container.

Use this transport when you want to amortize blob operations across many messages, avoid per-message queue charges, or when the queue service is unavailable in your environment. Expected end-to-end latency is bounded by `flush_interval_seconds`.

Auth is via an Azure Storage shared-key credential (legacy default) or an Azure AD Service Principal. See [Azure Service Principal Authentication](#) in the admin guide for the SP field reference, the minimum RBAC role (`Storage Blob Data Contributor`), and the no-auto-create rule in SP mode.

Message Lifecycle

1. Outbound appends each relay event to an in-memory WAL buffer.
2. Every `flush_interval_seconds` (default 60), the buffer is sealed and uploaded as a batch blob under a time-ordered key.
3. Inbound lists the container on a poll cycle and, for each batch blob, acquires a lease.
4. While the lease is held, messages are delivered to the inbound handler.
5. On success, the blob is deleted and the lease is released. On failure, the lease expires and another poller may retry.
6. If a lease grows older than `blob_lock_max_age_seconds` (default 300), it is considered stale and another inbound node may break the lease and take over.

Azure Blob Connection Configuration

Field	JSON Key	Required	Description
Service URL	<code>service_url</code>	Yes	Azure Blob Storage service URL. For US Government, use <code>*.blob.core.usgovcloudapi.net</code> .
Auth Mode	<code>auth_mode</code>	No	Either <code>shared-key</code> (the legacy default) or <code>service-principal</code> .
Azure Cloud	<code>azure_cloud</code>	No	Either <code>public</code> (default), <code>usgov</code> , or <code>china</code> . Selects the Azure AD authority and the data-plane SDK options when in Service Principal mode.
Account Name	<code>account_name</code>	Shared-key only	Azure Storage account name.
Account Key	<code>account_key</code>	Shared-key only	Azure Storage shared access key (base64, treat as secret). MUST be empty when <code>auth_mode</code> is <code>service-principal</code> .
Tenant ID	<code>tenant_id</code>	SP only	Azure AD tenant ID. Required when <code>auth_mode</code> is <code>service-principal</code> .
Client ID	<code>client_id</code>	SP only	Azure AD application (client) ID. Required when <code>auth_mode</code> is <code>service-principal</code> .
Client Secret	<code>client_secret</code>	SP only	Azure AD client secret value (treat as secret). Required when <code>auth_mode</code> is <code>service-principal</code> .
Blob Container Name	<code>blob_container_name</code>	Yes	Container holding both batch WAL blobs and deferred file blobs. Auto-created in shared-key mode; pre-provisioned in Service Principal mode.
Flush Interval	<code>flush_interval_seconds</code>	No	Seconds between WAL flushes on the outbound side. Default 60, minimum 5.
Blob Lock Max Age	<code>blob_lock_max_age_seconds</code>	No	Seconds before a stale lease may be broken on the inbound side. Default 300, minimum 30.

Field	JSON Key	Required	Description
Batch Poll Interval	<code>batch_poll_interval_secs</code>	No	Inbound poll cadence for new batch WAL blobs. Default 30, minimum 1. Lower values reduce end-to-end latency at the cost of more list operations.

Example: Azure Blob (shared-key)

```
{
  "name": "azure-blob-relay",
  "provider": "azure-blob",
  "file_transfer_enabled": true,
  "azure_blob": {
    "service_url": "https://mystorageaccount.blob.core.windows.net",
    "account_name": "mystorageaccount",
    "account_key": "base64-account-key",
    "blob_container_name": "crossguard-wal",
    "flush_interval_seconds": 60,
    "blob_lock_max_age_seconds": 300
  }
}
```

Example: Azure Blob (Service Principal)

```
{
  "name": "azure-blob-relay",
  "provider": "azure-blob",
  "file_transfer_enabled": true,
  "azure_blob": {
    "service_url": "https://mystorageaccount.blob.core.windows.net",
    "auth_mode": "service-principal",
    "azure_cloud": "public",
    "tenant_id": "000000000-0000-0000-0000-000000000000",
    "client_id": "11111111-1111-1111-1111-111111111111",
    "client_secret": "...",
    "blob_container_name": "crossguard-wal"
  }
}
```

Azure Service Bus Transport

Overview

The Azure Service Bus provider uses an AMQP-based Service Bus queue for message delivery and an optional Azure Blob Storage sidecar container for file transfer. Unlike the Azure Queue provider, Service Bus uses PeekLock receive semantics with server-side delivery counts and a native dead-letter queue (DLQ). Authentication is via a SAS connection string (legacy default) or an Azure AD Service Principal. In connection-string mode the blob sidecar takes its own shared-key credential separate from the Service Bus namespace; in Service Principal mode the blob sidecar inherits the parent SP credential so a single AAD identity covers both AMQP and blob access.

See [Azure Service Principal Authentication](#) in the admin guide for the SP field reference (including the additional `service_bus_namespace` required in SP mode), the minimum RBAC roles (`Azure Service Bus Data Sender` + `Azure Service Bus Data Receiver` on the queue, plus `Storage Blob Data Contributor` on the blob account when file transfer is enabled), and the credential-inheritance rule for the blob sidecar.

The plugin does *not* auto-create queues. Pre-create the queue in the Azure portal or via ARM/Terraform with your chosen `LockDuration` and `MaxDeliveryCount` before saving the connection. Lock duration is an entity property; the plugin cannot set it.

Message Flow

Outbound Server		Azure Service Bus		Inbound Server
+-----+		+-----+		+-----+
Serialize msg		Queue: crossguard-		ReceiveMessages
SendMessage	--->	relay (PeekLock)	--->	(batch 32)
				Complete on
		Blob Sidecar (opt):		handler success
Upload file?	--->	crossguard-files	--->	Abandon on error
Blob metadata		(Azure Blob Storage)		(15s blob poll)
+-----+		+-----+		+-----+

Receive Parameters

Parameter	Value	Description
Receive Mode	PeekLock	Messages are locked on receive; the plugin calls Complete on success or Abandon on handler error.
Receive Batch Size	32 messages	Maximum messages requested per receive call.
Receive Max Wait	5 seconds	Per-call context deadline. Empty-queue waits are bounded by this timeout, then the loop restarts.
Ack Timeout	10 seconds	Detached timeout for Complete / Abandon / Close so shutdown races cannot starve settlement.
Blob Poll Interval	15 seconds (configurable)	How often the inbound side polls the blob sidecar container for file attachments.
Default Max Message Size	192,000 bytes	Default ceiling for outbound payload size (Service Bus Standard tier caps messages at 256 KiB). Configurable up to 100 MiB on Premium.

Message Lifecycle

1. Outbound serializes the message envelope (JSON or XML) and sends it via the Service Bus SDK.
2. Oversized payloads (above `max_message_size_bytes`) are rejected at the client boundary before send.
3. Inbound receives messages in batches (up to 32) under PeekLock.
4. Messages with `DeliveryCount > 1` log a WARN (error code `26004`) so operators can detect poison-message buildup before DLQ moves happen.
5. Empty-body messages are dropped via Complete to avoid infinite redelivery loops (error code `26005`).
6. On handler success, the message is Completed. On handler error, the message is Abandoned; Service Bus increments `DeliveryCount` and redelivers.
7. After `MaxDeliveryCount` is exceeded, Service Bus auto-moves the message to the queue's dead-letter sub-queue. The plugin does NOT inspect or process DLQ messages; use the Azure portal, `az servicebus queue message` , or Azure monitoring to inspect the DLQ.

Service Bus File Transfer (Blob Sidecar)

When `file_transfer_enabled` is `true` and the blob sidecar fields are configured, file attachments are transferred via Azure Blob Storage using the same upload/download mechanics as the Azure Queue provider (`{post_id}/{unique_id}` blob keys, Base64-encoded JSON metadata under the `crossguard_headers` key). Service Bus queues themselves are message-only; files never flow through the queue.

Service Bus Connection Configuration

Field	JSON Key	Required	Description
Auth Mode	<code>auth_mode</code>	No	Either <code>connection-string</code> (the legacy default) or <code>service-principal</code> . Selects between the SAS-based connection string and Azure AD Service Principal authentication for the Service Bus namespace.
Connection String	<code>connection_string</code>	Connection-string mode only	SAS connection string for the Service Bus namespace. Required when <code>auth_mode</code> is <code>connection-string</code> ; MUST be empty when <code>auth_mode</code> is <code>service-principal</code> . Treat as a secret.
Azure Cloud	<code>azure_cloud</code>	No	Either <code>public</code> (default), <code>usgov</code> , or <code>china</code> . Selects the Azure AD authority. Data-plane routing uses <code>service_bus_namespace</code> directly, not <code>azure_cloud</code> .
Service Bus Namespace	<code>service_bus_namespace</code>	SP only	Fully-qualified Service Bus namespace, e.g. <code>myns.servicebus.windows.net</code> or <code>myns.servicebus.usgovcloudapi.net</code> . Required when <code>auth_mode</code> is <code>service-principal</code> because no SAS connection string encodes it.
Tenant ID	<code>tenant_id</code>	SP only	Azure AD tenant ID. Required when <code>auth_mode</code> is <code>service-principal</code> .

Field	JSON Key	Required	Description
Client ID	<code>client_id</code>	SP only	Azure AD application (client) ID. Required when <code>auth_mode</code> is <code>service-principal</code> .
Client Secret	<code>client_secret</code>	SP only	Azure AD client secret value (treat as secret). Required when <code>auth_mode</code> is <code>service-principal</code> .
Queue Name	<code>queue_name</code>	Yes	Pre-created queue name or hierarchical path. Up to 260 characters; letters, digits, periods, hyphens, underscores, and forward slashes. The plugin does NOT auto-create queues; operators must pre-create the queue with the desired <code>LockDuration</code> and <code>MaxDeliveryCount</code> .
Blob Service URL	<code>blob_service_url</code>	When file transfer enabled	Azure Blob Storage endpoint for the file sidecar.
Blob Account Name	<code>blob_account_name</code>	When file transfer enabled and sidecar uses shared-key	Storage account name for the blob sidecar (independent of the Service Bus namespace). MUST be empty when the parent <code>auth_mode</code> is <code>service-principal</code> (the SP credential is inherited by the sidecar; mixed-mode sidecars are not supported).
Blob Account Key	<code>blob_account_key</code>	When file transfer enabled and sidecar uses shared-key	Shared access key for the blob sidecar storage account. Treat as a secret. MUST be empty when the parent <code>auth_mode</code> is <code>service-principal</code> .
Blob Container Name	<code>blob_container_name</code>	When file transfer enabled	Blob container used for file attachments. Auto-created on startup in shared-key mode; pre-provisioned in Service Principal mode.
Max Message Size	<code>max_message_size_bytes</code>	No	Outbound payload ceiling. Default 192000. Range 1024 to 104857600 (100 MiB Premium cap).

Field	JSON Key	Required	Description
Blob Poll Interval	blob_poll_interval_seconds	No	Inbound blob-sidecar poll cadence. Default 15, minimum 1.

Example: Service Bus (SAS connection string)

```
{
  "name": "azure-servicebus-relay",
  "provider": "azure-servicebus",
  "file_transfer_enabled": true,
  "azure_servicebus": {
    "connection_string": "Endpoint=sb://myns.servicebus.windows.net/;SharedAccessKeyName=RootManageSharedAccessKey;SharedAccessKey=...",
    "queue_name": "crossguard-relay",
    "blob_service_url": "https://mystorageaccount.blob.core.windows.net",
    "blob_account_name": "mystorageaccount",
    "blob_account_key": "base64-account-key",
    "blob_container_name": "crossguard-files"
  }
}
```

Example: Service Bus (Service Principal)

```
{
  "name": "azure-servicebus-relay",
  "provider": "azure-servicebus",
  "file_transfer_enabled": true,
  "azure_servicebus": {
    "auth_mode": "service-principal",
    "azure_cloud": "public",
    "service_bus_namespace": "myns.servicebus.windows.net",
    "tenant_id": "00000000-0000-0000-0000-000000000000",
    "client_id": "11111111-1111-1111-1111-111111111111",
    "client_secret": "...",
    "queue_name": "crossguard-relay",
    "blob_service_url": "https://mystorageaccount.blob.core.windows.net",
    "blob_container_name": "crossguard-files"
  }
}
```

```
}  
}
```

Loop Prevention and Delivery Guarantees

Cross Guard rides on the shared-channels framework's intrinsic loop prevention (every received user carries a non-empty `RemoteId` ; the framework refuses to re-sync content marked as remote) and adds three plugin-level guards:

- **Per-remote outbound suppression.** The outbound hook resolves the target connection via `connNameForRemote(rc.RemoteId)` and refuses to send back to the remote a message originated from. A sync envelope that arrived from connection A never re-leaves on connection A.
- **Connection linkage validation.** Envelopes are only published to outbound connections that are explicitly linked to the originating team. Inbound envelopes are only processed if the target team and channel are linked to the receiving inbound connection; otherwise the inbound handler posts an Accept / Block prompt rather than applying the content.
- **Inbound deduplication.** The inbound sequencer keys on `(Epoch, ConnName, ChannelId, Sequence)` and drops any envelope it has already admitted. Combined with the single-active-receiver lease (only one node per cluster subscribes at a time), this catches duplicates from at-least-once providers and replays from a sender restart.

System messages (channel join / leave, header changes, etc.) ride the `SyncMsg.MembershipChanges` container or as system-typed Posts; both are relayed normally and routed by the framework on the receiving side. The plugin does not filter them out, since the framework needs the events to keep its synced channel membership view current.

OpenAPI Specification

The machine-readable OpenAPI 3.1 specification for this interface is available at `schema/crossguard-api.yaml` in the plugin source repository.